

DESIGN REPORT

Project 12 | CSE406 — Cyber Security Sessional

ICMP Blind Connection-Reset and Blind Throughput-Reduction Attack against TCP

SUBMITTED BY

Debashri Roy

2105035

Anisa Binte Asad

2105036

September 7, 2026

Contents

1	Overview of the Project Idea	2
2	Network Topology, Components and Timing Diagram	3
2.1	Topology Diagram	3
2.2	Components	3
2.3	Timing Diagram — Attack 1 (Blind Reset)	4
2.4	Timing Diagram — Attack 2a (Blind Throughput Reduction via PMTU)	4
2.5	Timing Diagram — Attack 2b (Congestion-Window Shrink via Source Quench)	5
3	Packet and Frame Structure	5
3.1	Spoofed ICMP Packet (both attacks) — general structure	5
3.2	Attack 1 — ICMP fields used	6
3.3	Attack 2 — ICMP fields used	6
4	Implementation Plan	6
5	Expected Outcome of a Successful Attack	8
6	Defense Ideas	8

1 Overview of the Project Idea

This project studies two related off-path, blind attacks on TCP connections that exploit how TCP stacks react to spoofed ICMP error messages. Both attacks rely on the weakness class described in RFC 5927 (*ICMP Attacks against TCP*) [1]: an attacker who is not on the path between two TCP endpoints, and who cannot sniff the connection's real sequence numbers, injects spoofed ICMP error packets that cause the victim's TCP stack to take a damaging action.

Attack 1 — Blind Connection-Reset via ICMP

Historically, TCP stacks could terminate a connection upon receiving an ICMP *Destination Unreachable* error (Type 3, Code 2/3/4) that appeared to reference the connection. An off-path attacker can spoof such a packet by guessing the connection's 4-tuple and embedding a plausible TCP header, without observing the actual traffic. Modern kernels mitigate this attack by checking whether the embedded TCP sequence number falls within the current window before processing the ICMP message [1].

Attack 2 — Blind Throughput-Reduction via ICMP

Instead of an outright reset, the attacker spoofs *soft errors* that do not kill the connection but severely degrade it. Because soft errors are not subject to the same strict in-window sequence validation as hard errors, these attacks remain practical even on modern, patched kernels. The attacker can degrade throughput using two main strategies:

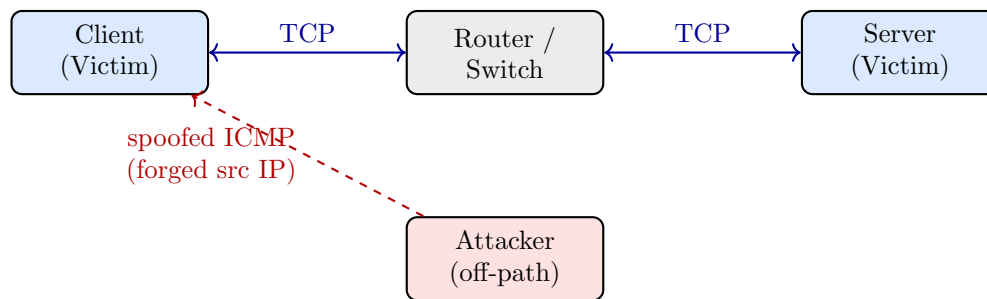
- **Forcing PMTU Reduction:** The attacker spoofs ICMP *Fragmentation Needed* (Type 3, Code 4) messages used for Path MTU Discovery (PMTUD). This forces the sender to drastically shrink its assumed MTU (Maximum Transmission Unit, the largest packet size that can be transmitted over the network without being fragmented), causing excessive fragmentation and increased per-byte overhead [1].
- **Shrinking the Congestion Window (cwnd):** The `cwnd` dictates how much unacknowledged data TCP is allowed to have in transit. Historically, an attacker could spoof an ICMPv4 Source Quench (Type 4, Code 0) message to force the sender's congestion control into slow start, triggering TCP Reno's multiplicative decrease and shrinking a large `cwnd` abruptly [2]. **Note:** Linux has ignored ICMPv4 Source Quench since 2004 [1], so this vector is treated as secondary and will only be reported as a working attack if verified against our own setup; PMTU reduction above is the primary throughput-degradation mechanism for this project.

Kernel strategy

Because modern kernels validate the embedded sequence number for hard errors [1], the **Attack 1** demonstration uses a modified kernel that relaxes this specific check (branch `custom.5.15`) [5], so the underlying vulnerability class can be shown end-to-end in a controlled lab. **Attack 2** does not need to defeat that check, since soft-error handling is not gated the same way, so it will be built and demonstrated against a **stock, unmodified kernel** [4]. Both kernels will additionally be tested against the opposite attack, to record the contrast (Attack 1 vs. stock kernel: expected failure; Attack 2 vs. patched kernel: expected to still succeed, since nothing about the patch affects soft-error handling). The assumed congestion-control algorithm for reasoning about Attack 2's effect on throughput is **TCP Reno** [2].

2 Network Topology, Components and Timing Diagram

2.1 Topology Diagram

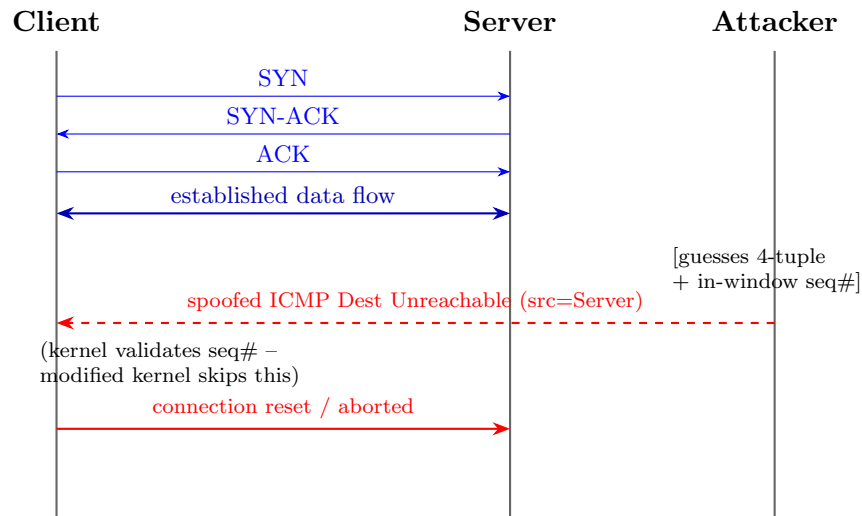


2.2 Components

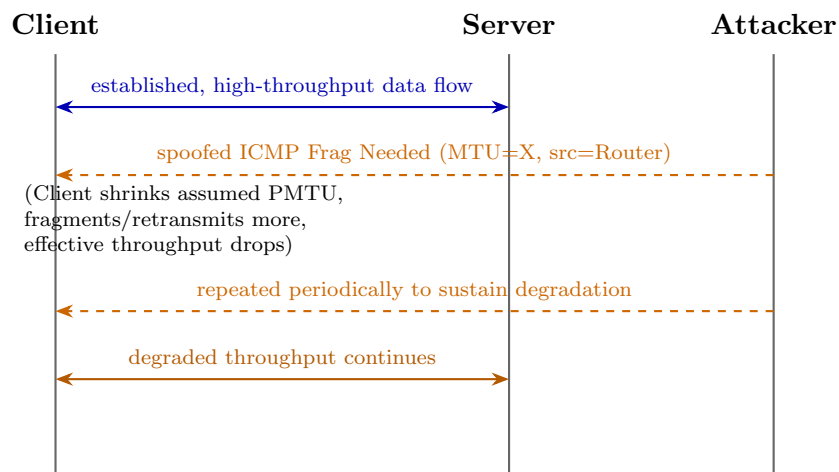
Setup: Client, Server, and Attacker run as three isolated Linux network namespaces on a single machine, connected using virtual ethernet (`veth`) pairs and a virtual bridge, instead of full VMs. This still gives each host its own network stack and controllable routes, and keeps the Attacker off the Client–Server segment so it can’t ARP-spoof or sniff that traffic without the disk and memory cost of three separate VMs. This isolation only works if the topology is built correctly, with the Attacker’s interface kept off the Client–Server segment; it isn’t automatic just from using namespaces. Since namespaces share the host’s kernel, the custom kernel needed for Attack 1 is built as the machine’s actual boot kernel rather than a per-namespace one.

Role	Description
Client (Victim A)	Initiates a long-lived TCP connection (bulk transfer / <code>iperf3</code> session) to the Server.
Server (Victim B)	Accepts the TCP connection, sends/receives bulk data.
Off-path Attacker	Has no interception capability on the Client↔Server path. Cannot see real packets of the victim connection. Can only send spoofed packets and must infer/guess the 4-tuple (and for Attack 1, an in-window sequence number).
Router/Switch(es)	Standard L2/L3 forwarding between the three namespaces; establishes that attacker traffic arrives via a separate path/interface, preserving the “off-path” assumption.

2.3 Timing Diagram — Attack 1 (Blind Reset)

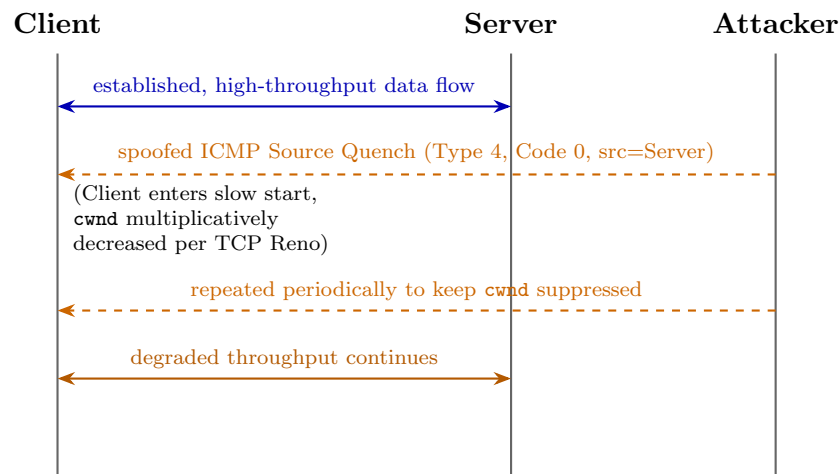


2.4 Timing Diagram — Attack 2a (Blind Throughput Reduction via PMTU)



2.5 Timing Diagram — Attack 2b (Congestion-Window Shrink via Source Quench)

Secondary vector — effective only if the target kernel still honors ICMPv4 Source Quench not guaranteed on modern Linux.



3 Packet and Frame Structure

3.1 Spoofed ICMP Packet (both attacks) — general structure

Ethernet Header
IP Header src = spoofed (Router/Server) dst = Victim (Client) proto = 1 (ICMP)
ICMP Header type / code (see §3.2, §3.3) checksum unused/pointer (type-specific) ← Attack 2a (Frag-Needed, Type 3/Code 4): carries forged Next-Hop MTU Attack 2b (Source Quench, Type 4/Code 0): field is genuinely unused — arrival of the message itself triggers the cwnd cut
Embedded (offending) packet Original IP header : src = Client dst = Server proto = 6 (TCP) Original TCP header (8B+) : src port, dst port sequence number ← must be in-window for stack to act (hard errors only)

3.2 Attack 1 — ICMP fields used

- **Type 3** (Destination Unreachable), **Code 2** (Protocol Unreachable) / **Code 3** (Port Unreachable) — hard errors historically mapped by BSD-derived stacks to `ECONNRESET`/abort.
- Embedded IP+TCP header must match the real connection's 4-tuple (attacker must guess/brute-force the source port if unknown; the destination port is often known, e.g. a well-known service).
- Embedded TCP sequence number: compliant stacks (post-patch) require this to fall within `SND.UNA`, `SND.NXT` (where `SND.UNA` is the oldest unacknowledged sequence number and `SND.NXT` is the next sequence number to be sent) before honoring the error [1]. This is the check the provided modified kernel relaxes for demonstration [5].

3.3 Attack 2 — ICMP fields used

- **Type 3** (Destination Unreachable), **Code 4** (Fragmentation Needed and DF set) — used for PMTUD; carries a Next-Hop MTU field.
- Spoofing this with an artificially small MTU value (e.g., 68–500 bytes) causes the sender to shrink its effective MSS, increasing per-byte overhead and triggering more retransmissions/timeouts if done aggressively, thereby reducing goodput.
- Alternative vector: spoofed ICMPv4 Source Quench (**Type 4, Code 0**) induces a perceived congestion signal, causing TCP Reno's AIMD behavior to cut `cwnd` (multiplicative decrease) ties directly into the congestion-control dynamics of TCP Reno [2].
- The embedded TCP header again must match the victim connection's 4-tuple, but no in-window sequence-number check applies to soft errors the same way it does to hard-error handling which is why this remains effective against modern kernels. This should be verified empirically against the specific kernel version tested and documented precisely in the final report.

4 Implementation Plan

Phase 1 — Environment Setup

1. Set up 3 Linux network namespaces: Client, Server, Attacker, connected via veth pairs and a virtual bridge, with the topology built so the Attacker is genuinely off-path (no visibility into the Client–Server segment).
2. Build and boot the modified kernel (`custom_5.15` branch) as the host's boot kernel for the Attack 1 demonstration since namespaces share the host kernel, this requires rebooting into `custom_5.15` for Attack 1's run and back into the stock kernel for Attack 2 and baseline/before-after comparison rather than running both simultaneously. Alternatively, since we have two laptops available, one can be dedicated to the modified kernel (Attack 1) and the other kept on the stock kernel (Attack 2 and baseline), avoiding the need to reboot between demonstrations entirely.

Phase 2 — Baseline Traffic

3. Establish a long-lived TCP bulk transfer (`iperf3`, or `scp/curl` of a large file) between Client and Server.
4. Capture baseline throughput/latency with `tcpdump`/Wireshark and `ss -i` (or `iperf3` reporting) with no attack running, for comparison.

Phase 3 — 4-Tuple / Sequence Inference (Off-path Constraint)

5. Model the “blind” aspect explicitly: assume the attacker knows the Client and Server IPs and the Server’s well-known port, but must guess the Client’s ephemeral source port (and, for Attack 1, an in-window sequence number). Implement a brute-force/guessing strategy over the plausible port range and discuss the packet-rate cost (tie this to RFC 5927 §5–6 [1], which analyze the number of packets needed for a given success probability).

Phase 4 — Attack 1: Blind Reset

6. On the modified-kernel host, craft and send spoofed ICMP Destination Unreachable packets targeting the guessed 4-tuple.
7. Observe and record connection termination (via `ss`, application-level error, packet capture showing the connection torn down) with no ICMP reaching the Client through any legitimate/in-path device.

Phase 5 — Attack 2: Blind Throughput Reduction

8. On the stock-kernel host, craft and send spoofed ICMP Fragmentation-Needed packets with a reduced MTU value, at intervals, during an active bulk transfer (Attack 2a).
9. Measure resulting throughput degradation over time and correlate with observed MTU/MSS changes and retransmission counts on the Client.
10. On the same stock-kernel host, craft and send spoofed ICMPv4 Source Quench packets at intervals during an active bulk transfer (Attack 2b), and verify beforehand whether the kernel still honors Source Quench, since this vector is inactive by default on modern Linux.
11. Measure resulting `cwnd`/throughput changes via `ss -i`, and record whether the attack had any measurable effect, given the empirical uncertainty noted above.

Phase 6 — Data Collection & Analysis

12. Repeat each attack multiple times with varying guess-space sizes / attack packet rates to build success-probability vs. packet-count curves (as RFC 5927 analyzes theoretically).
13. Collect packet captures, kernel logs (`dmesg`), and throughput graphs as evidence for the report.

Phase 7 — Defense Prototype

14. Implement/validate the defense idea(s) from Section 6 and re-run the attacks to show mitigation.

Tools

- Scapy (crafting spoofed IP/ICMP/TCP packets) *or* hand-rolled raw sockets per the “no ready-made attack tool” constraint — confirm which is permitted
- `iperf3/netperf` (traffic generation)
- Wireshark/`tcpdump` (capture & verification)
- `ss/netstat` (connection state)
- Custom kernel build (`custom_5.15` branch)
- Python/bash scripts for orchestration and logging

5 Expected Outcome of a Successful Attack

Attack 1 (Blind Reset)

- The Client's TCP connection to the Server transitions abruptly to **CLOSED**, visible via `ss -ta` no longer listing the connection, and the application receiving **ECONNRESET/ECONNREFUSED**-style errors, despite no legitimate reset/FIN having been sent by either real endpoint.
- Packet captures on the Client showing only the spoofed ICMP packet (no corresponding real ICMP from an actual on-path router or from the Server) coinciding with connection teardown.
- On an unmodified kernel, the same spoofed packet is expected to be silently dropped/ignored by the sequence-validation check demonstrating the protection's effect (the "before vs. after kernel patch" comparison).

Attack 2 (Blind Throughput Reduction)

- Measurable, sustained drop in observed throughput (e.g., `iperf3`-reported Mbps) during the attack window compared to baseline.
- **Attack 2a (PMTU)**: evidence of reduced effective MSS/MTU on the Client after the spoofed ICMP Frag-Needed message.
- **Attack 2b (Source Quench)**: evidence of `cwnd/ssthresh` reduction (via `ss -i` extended TCP info, or kernel tracepoints) consistent with TCP Reno's AIMD response contingent on the kernel still honoring Source Quench, per the caveat above.
- The connection remains alive (unlike Attack 1) the goal is degradation, not termination making this attack stealthier and harder to detect at the application layer.

6 Defense Ideas

Attack 1 — covered by the planned before/after comparison

The sequence-in-window check in `tcp_v4_err()` is the defense. Phase 4 will test the same spoofed ICMP packet on both kernels: the custom kernel (check relaxed, attack expected to succeed) and the stock kernel (check active, attack expected to fail). This comparison shows whether the defense works.

Attack 2a — Defense Plan: PMTU Floor

Primary defense (`min_pmtu floor`): We set `net.ipv4.route.min_pmtu` from its default of 552 B to 1200 B on the Client, preventing PMTUD from accepting spoofed Frag-Needed values below this floor. In Phase 7, we re-send the same Attack 2a packet from Phase 5 (spoofed Next-Hop MTU = 68 B) and compare throughput, MSS, and retransmissions with the Phase 5 baseline. This single-sysctl mitigation is directly testable and is our guaranteed deliverable.

Known limitation: This only limits the damage. An attacker can still spoof an MTU just above the floor (e.g. 1201 B) and reduce the PMTU, though less severely. The underlying issue of trusting unauthenticated ICMP remains.

Stretch goal (time permitting): PLPMTUD (RFC 4821) [3]: PLPMTUD removes ICMP from the trust path by using TCP data probes to determine the path MTU. Linux supports this via `net.ipv4.tcp_mtu_probing=1`. We leave this outside the guaranteed scope because proper validation requires checking probe sizes, ACKs, and legitimate PMTU changes. If time permits

after Phases 1–7, we will enable it, re-run Attack 2a, and verify that legitimate PMTU changes are still detected.

References

1. RFC 5927 — ICMP Attacks against TCP (Sections 5 and 6 especially)
2. RFC 5681 — TCP Congestion Control
3. RFC 4821 — Packetization Layer Path MTU Discovery
4. Linux kernel source: `net/ipv4/tcp_ipv4.c`, `net/ipv4/icmp.c`
5. Reference kernel: `custom_5.15` branch